DataBridge V2

From PostgreSQL Schema to Production-Ready API

Automated Backend Code Generation Platform

Maple Technologies -- Engineering Division

Agenda

1. Architecture Overview and Design Philosophy
2. Configuration System
3. Parser Engine and Schema Introspection
4. Template System and Code Generation
5. Generated Code Walkthrough
6. Pagination Engine
7. Real-Time Events and SSE
8. Documentation Engine
9. Testing and Quality Assurance
10. Performance Engineering
11. Live Demo Showcase
12. Summary and Roadmap

Executive Summary

DataBridge V2 is a schema-driven code generation platform that transforms PostgreSQL database schemas into fully operational REST APIs.

What It Generates

- **Go Structs** with GORM tags, validation, and JSON serialization
- **Service Layer** with CRUD, bulk operations, and transaction support
- **HTTP Controllers** with Swagger annotations and error handling
- **Route Registration** with recursive URL tree grouping
- **Unit and Integration Tests** with mock infrastructure
- **Event Handlers** with sync/async dispatch
- **RPC Wrappers** for PostgreSQL stored procedures
- **API Documentation** (VitePress, OpenAPI, MCP, Training Data)

Key Value Proposition

Before DataBridge V2

- Manual struct definition from SQL schemas
- Hand-written CRUD services with inconsistent patterns
- Copy-paste controller boilerplate across endpoints
- Swagger annotations maintained separately from code
- No standardized test generation

After DataBridge V2

- **One command** generates the entire API layer
- Consistent patterns across all tables and schemas
- Swagger docs embedded in generated code
- Tests generated alongside business logic
- Real-time event hooks and SSE built in

Technical Stack

Layer	Technology
Language	Go 1.25+
HTTP Framework	Fiber (Express-style, built on fasthttp)
ORM	GORM with PostgreSQL driver
Database	PostgreSQL 14+ (with pgvector, PostGIS support)
Documentation	Swagger/OpenAPI 3.x, VitePress
Template Engine	Go `text/template` with 40+ custom functions
Serialization	bytedance/sonic (high-performance JSON)
Testing	Go testing + httptest + testify
CI/CD	GitHub Actions

Project Scale

Metric	Value
Core Source Files	80+ Go files
Template Files	13 code generation templates
Pagination Engine	30+ files, 3000+ LOC
Template Functions	40+ custom FuncMap entries
Supported SQL Types	25+ PostgreSQL types mapped to Go
Doc Generator Outputs	6 formats (MD, OpenAPI, MCP, TS, Training, Mermaid)
Test Coverage	Unit, Integration, E2E, Benchmark
Generated Output per Table	6-8 files (struct, service, controller, tests, events)

Architecture Overview

Design Philosophy

DataBridge V2 follows three core principles:

1. **Schema-First Development** -- The database schema is the single source of truth.

System Architecture

```
+-----------------------------------------------------------------+
|                      CLI / Embedded Entry                       |
|  cmd/databridge-v2/main.go  <-->  app/app.go + app/run.go       |
|                      | reads config.yaml                        |
|             app/config.go (Config, Project, SchemaConfig)       |
+-----------------------------------------------------------------+
                       |
                       | creates GeneratorContext
                       | opens PostgreSQL via db/db.go
                       v
+-----------------------------------------------------------------+
|                   parser/creator.go                             |
|  Per Schema:                                                    |
|  1. GenerateEnumTypes()     -> enums.go                         |
|  2. GetTables()             -> SQL introspection                |
|  3. GetRPCFunctions()       -> rpc__ function discovery         |
|  4. Per Table (parallel):                                       |
|     - StructCreator         -> structures/{table}.go            |
|     - ServiceCreator        -> services/{table}.go              |
|     - ControllerCreator     -> controllers/{table}.go           |
|     - TestCreator           -> tests/{table}_test.go            |
|  5. EndPointCreator         -> {schema}/{schema}.go             |
|  6. EventSetupCreator       -> event_handlers/                  |
|  7. RPCCreator              -> rpc services + controllers       |
+-----------------------------------------------------------------+
```

Layered Architecture

The generated code follows a strict **three-layer architecture**:

```
  HTTP Request
       |
       v
+-----------------+
|   Controllers   |  Fiber handlers, request parsing,
|                 |  Swagger annotations, response formatting
+-----------------+
       |
       v
+-----------------+
|    Services     |  Business logic, GORM queries,
|                 |  transaction management, outbox
+-----------------+
       |
       v
+-----------------+
|    Structures   |  Go structs (Form, Model, Edit,
|                 |  Filter, Identity, Page variants)
+-----------------+
       |
       v
+-----------------+
|   PostgreSQL    |  Tables, ENUMs, Views, RPC Functions
+-----------------+
```

Package Dependency Graph

```
                app/
               /    \
        config.go   run.go
             |        |
             v        v
      generator/   parser/
      context.go   creator.go
                     |
         +-----------+-----------+
         |           |           |
         v           v           v
    struct_     service_    controller_
    creator     creator     creator
         |           |           |
         +-----------+-----------+
                     |
         +-----------+-----------+
         |           |           |
         v           v           v
    models/     helpers/    templates/
    (data)     (utils)     (.go.tmpl)
```

All creators share the same `GeneratorContext` instance -- no cross-creator coupling.

Module Boundary Design

Generator-Time Packages (Build Dependencies)

Package	Responsibility
`app/`	Orchestration, config loading, project management
`parser/`	Schema introspection, template execution, file output
`generator/`	GeneratorContext definition
`models/`	Table, Column, Constraint, RPC data structures
`helpers/`	Type conversion, struct tags, file I/O
`templates/`	Go template files (.go.tmpl)

Runtime Packages (Used by Generated Code)

Package	Responsibility
`pagination/`	Generic query engine with filtering, sorting, caching
`sse/`	Server-Sent Events hub with backplane support
`packages/events/`	Route-scoped event manager with sync/async dispatch

Data Flow: Schema to API

```
PostgreSQL                DataBridge V2              Generated API
+-------------+         +----------------+         +------------------+
| Schema      | ------> | Introspection  | ------> | Go Structs       |
| Tables      |   SQL   | (info_schema)  |  tmpl   | (6 variants)     |
| Columns     |         +----------------+         +------------------+
| Constraints |                |                          |
| FKs         |                v                          v
| ENUMs       |         +----------------+         +------------------+
| Views       |         | Type Mapping   |         | GORM Services    |
| Functions   |         | Tag Generation |         | (CRUD + Bulk)    |
+-------------+         +----------------+         +------------------+
                               |                          |
                               v                          v
                        +----------------+         +------------------+
                        | Template       |         | Fiber Controllers|
                        | Execution      |         | (Swagger + Auth) |
                        +----------------+         +------------------+
                               |                          |
                               v                          v
                        +----------------+         +------------------+
                        | File Writer    |         | Route Tree       |
                        | (Buffered I/O) |         | (Recursive Group)|
                        +----------------+         +------------------+
```

Multi-Schema Support

DataBridge V2 processes multiple PostgreSQL schemas in a single run.

Each schema produces an isolated Go package with its own routes.

```
config.yaml
  projects:
    - name: "my-project"
      schemas:
        - name: "iam"          ->  api_v2/iam/
        - name: "catalog"      ->  api_v2/catalog/
        - name: "orders"       ->  api_v2/orders/
        - name: "logistics"    ->  api_v2/logistics/
        - name: "analytics"    ->  api_v2/analytics/
```

Cross-schema FK references are resolved via a shared `shared_types.go` file generated by querying `information_schema.referential_constraints`.

The main `api.go` aggregation file wires all schema routers into a single Fiber application with a unified route prefix.

Configuration System

YAML-Driven Code Generation

Project Configuration

Each project defines its output target, module identity, and schema list.

```yaml
projects:
  - name: "innovation-hub"
    projectPath: "github.com/maple-tech/backend-generator/apiv2/data-bridge-examples"
    moduleName: "data-bridge-examples"
    outputFolder: "api_v2"
    generateTests: true
    sseEnabled: true
    outboxEnabled: true
    eventManagerIntegration: true
    schemas:
      - name: "iam"
      - name: "catalog"
      - name: "orders"
      - name: "logistics"
        relaxGrouping: true
      - name: "analytics"
```

Schema-Level Configuration

Fine-grained control per schema:

Option	Type	Purpose
`name`	string	PostgreSQL schema name
`skipSchemaPrefixInOps`	bool	Remove schema prefix from operation names
`ignoreTables`	[]string	Exclude specific tables from generation
`relaxGrouping`	bool	Flatten route tree instead of deep nesting
`eventHandlers.generate`	bool	Generate event handler stubs
`eventHandlers.outputDir`	string	Custom output directory for handlers
`eventHandlers.packageName`	string	Go package name for handlers

Feature Flags

The configuration supports granular feature toggles at both the global level and per-project override level.

```
                Global Defaults
                      |
          +-----------+-----------+
          |           |           |
      Project A   Project B   Project C
      (override)  (default)   (override)
```

Flag	Effect When Enabled
`generateTests`	Produces unit tests, integration tests, test helpers
`rbacEnabled`	Adds permission middleware to route registration
`sseEnabled`	Generates SSE stream endpoints and hub initialization
`outboxEnabled`	Adds transactional outbox pattern to service creates/updates
`eventManagerIntegration`	Wires EventManager hooks into controller lifecycle

Project-level flags use pointer types (`*bool`) to distinguish "not set" from "explicitly false", enabling clean inheritance.

GeneratorContext

All configuration resolves into a single immutable struct passed through the entire generation pipeline.

```
type GeneratorContext struct {
    ProjectName           string
    ProjectPath           string
    OutputFolder          string
    ProjectImportPath     string
    HelperLibBasePath     string
    PaginationPackagePath string
    ImportRepoPath        string
    SSEPackagePath        string

    GenerateTests  bool
    RBACEnabled    bool
    SSEEnabled     bool
    OutboxEnabled  bool
}
```

No global state. Every creator, helper, and template function receives this context explicitly, enabling safe concurrent execution.

Configuration Validation

The system validates configuration at startup before any generation begins:

1. **Project name** must be non-empty
2. **Output folder** must be specified
3. **At least one schema** must be defined per project
4. **Database connection** (DSN) must be valid and reachable
5. **Import paths** must be resolvable within the Go module system

Invalid configuration produces clear error messages with the specific field and project that failed validation -- not cryptic template errors downstream.

Configuration to Output Mapping

```
config.yaml                        Generated Output
+-------------------------+     +-----------------------------+
| moduleName: "my-app"     | --> | go.mod: module my-app       |
| outputFolder: "api_v2"   | --> | api_v2/                     |
| schemas:                  |     |   iam/                       |
|   - name: "iam"          | --> |     structures/              |
|     ignoreTables:         |     |     services/                |
|       - "migrations"      |     |     controllers/             |
|   - name: "catalog"      | --> |   catalog/                   |
|     relaxGrouping: true   |     |     structures/              |
|     skipSchemaPrefixInOps | --> |     (flat route tree)        |
| sseEnabled: true          | --> |   sse/ stream endpoints      |
| outboxEnabled: true       | --> |   outbox/ event files        |
| generateTests: true       | --> |   tests/ + test_helpers/     |
+-------------------------+     +-----------------------------+
```

Every configuration option has a direct, traceable impact on the generated output. No hidden behavior or implicit conventions.

Parser Engine

Schema Introspection Pipeline

The parser is the heart of DataBridge V2. It connects to PostgreSQL, reads the schema metadata, and drives the code generation pipeline

Schema Introspection

The `GetTables()` function executes a sophisticated multi-CTE SQL query against PostgreSQL `information_schema`:

```
WITH table_columns AS (
    SELECT c.table_name, c.column_name, c.data_type,
           c.is_nullable, c.udt_name, c.column_default,
           pgd.description AS comment
    FROM information_schema.columns c
    LEFT JOIN pg_catalog.pg_description pgd ...
    WHERE c.table_schema = $1
),
table_constraints AS (
    SELECT tc.constraint_type, tc.constraint_name,
           kcu.column_name, tc.table_name
    FROM information_schema.table_constraints tc
    JOIN information_schema.key_column_usage kcu ...
),
foreign_keys AS (
    SELECT kcu.column_name, ccu.table_name AS foreign_table,
           ccu.column_name AS foreign_column
    FROM information_schema.referential_constraints rc ...
)
SELECT ... FROM table_columns
JOIN table_constraints ...
LEFT JOIN foreign_keys ...
```

Data Model: Table and Column

Every introspected table is represented by rich data structures:

```go
type Table struct {
    Name         string
    Schema       string
    Type         string           // "BASE TABLE", "VIEW"
    Columns      TableColumns     // []TableColumn
    PrimaryKeys  TableConstraints // composite PK support
    ForeignKeys  TableConstraints
    UniqueKeys   TableConstraints
}

type TableColumn struct {
    Name          string
    Type          string           // SQL data type
    Nullable      string           // "YES" / "NO"
    UdtName       string           // PostgreSQL user-defined type
    Default       *string          // column default expression
    Comment       *string          // pg_description
    Constraints   TableConstraints
    RelatedTables TableRelateds    // FK targets
}
```

Slice types implement `database/sql.Scanner` for GORM JSONB compatibility.

RPC Function Discovery

DataBridge V2 automatically discovers PostgreSQL stored procedures that follow the `rpc__` naming convention:

```
SELECT p.proname, n.nspname,
       pg_get_function_arguments(p.oid),
       pg_get_function_result(p.oid)
FROM pg_proc p
JOIN pg_namespace n ON p.pronamespace = n.oid
WHERE n.nspname = $1
  AND p.proname LIKE 'rpc__%'
```

Each discovered function becomes:

- A typed Go struct for parameters
- A service method calling `db.Raw()`
- A Fiber controller with Swagger annotations
- A registered endpoint on the schema router

Generation Pipeline

```
Schema Name
     |
     v
+------------------+
| GenerateEnumTypes|  Query pg_enum -> enums.go
+------------------+
     |
     v
+------------------+
| GetTables()      |  Query information_schema -> []Table
+------------------+
     |
     v
+------------------+
| GetRPCFunctions()|  Query pg_proc -> []RPCFunction
+------------------+
     |
     v  (for each table, optionally parallel)
+------------------+     +------------------+
| StructCreator    | --> | structures/      |
+------------------+     +------------------+
| ServiceCreator   | --> | services/        |
+------------------+     +------------------+
| ControllerCreator| --> | controllers/     |
+------------------+     +------------------+
| TestCreator      | --> | tests/           |
+------------------+     +------------------+
     |
     v
+------------------+     +------------------+
| EndPointCreator  | --> | {schema}.go      |
+------------------+     +------------------+
| EventSetupCreator| --> | event_handlers/  |
+------------------+     +------------------+
| RPCCreator       | --> | rpc_service/ctrl |
+------------------+     +------------------+
```

Enum Type Generation

PostgreSQL ENUMs are introspected and mapped to typed Go constants:

Input (PostgreSQL):

```sql
CREATE TYPE iam.user_status AS ENUM (
    'active', 'inactive', 'suspended', 'pending'
);
```

Output (Go):

```go
type UserStatus string

const (
    UserStatusActive    UserStatus = "active"
    UserStatusInactive  UserStatus = "inactive"
    UserStatusSuspended UserStatus = "suspended"
    UserStatusPending   UserStatus = "pending"
)
```

All enum values are type-safe. The generator validates that column references to enum types produce the correct Go type in struct fields.

Recursive Route Tree Grouping

The `groupTablesIntoRoutes()` function builds a hierarchical URL tree from PostgreSQL table naming conventions:

Input Tables:

```
users, user_settings, user_preferences, user_notifications,
products, product_variants, product_variant_prices
```

Generated Route Tree:

```
/users
  GET    /          -> Search
  POST   /          -> Create
  GET    /:id       -> Find
  PUT    /:id       -> Update
  DELETE /:id       -> Delete
  /settings
    GET  /          -> Search user_settings
    ...
  /preferences
    GET  /          -> Search user_preferences
    ...
  /notifications
    GET  /          -> Search user_notifications
    ...
/products
  /variants
    /prices         -> product_variant_prices
```

Creator Interface Pattern

All creators follow the same structural pattern:

```go
type StructCreator struct {
    DB      *gorm.DB
    Context *generator.GeneratorContext
    // creator-specific fields
}

func (sc *StructCreator) CreateTableStruct(
    table models.Table,
    schemaName string,
) error {
    // 1. Prepare template data
    data := map[string]interface{}{
        "Table":     table,
        "Schema":    schemaName,
        "Context":   sc.Context,
        "Imports":   sc.resolveImports(table),
    }

    // 2. Execute template
    output := templateCache.Execute("struct.go.tmpl", data)

    // 3. Write to disk
    return helpers.WriteFileToDisk(path, output)
}
```

This consistency makes the system easy to extend:

adding a new output type means adding a new creator + template.

Template Cache Architecture

The `TemplateCache` is a performance-critical singleton that pre-loads and caches all templates at startup.

```go
type TemplateCache struct {
    templates map[string]*template.Template
    mu        sync.RWMutex
}

var (
    globalCache *TemplateCache
    cacheOnce   sync.Once
)

func GetTemplateCache() *TemplateCache {
    cacheOnce.Do(func() {
        globalCache = &TemplateCache{
            templates: make(map[string]*template.Template),
        }
        globalCache.loadAll()
    })
    return globalCache
}
```

Key design decisions:

- `sync.Once` ensures thread-safe lazy initialization
- Templates are **cloned** before execution to avoid shared `define` block conflicts
- **Hot-reload** support for development: detects file changes and reloads

Custom FuncMap -- Naming Conventions

Functions that transform SQL identifiers into Go conventions:

Function	Input	Output
`ToCamel`	`user_settings`	`UserSettings`
`ToLowerCamel`	`user_settings`	`userSettings`
`ToSnake`	`UserSettings`	`user_settings`
`Plural`	`category`	`categories`
`Singular`	`categories`	`category`
`SafeFieldName`	`type`	`TypeField` (avoids Go keywords)
`TrimSchemaPrefix`	`iam_users`	`users` (with schema context)

These ensure consistent naming across all generated files without manual intervention or naming configuration.

Custom FuncMap -- Type System

Functions that map PostgreSQL types to Go types:

```go
// GoType returns the Go type string for a column
func GoType(col models.TableColumn) string {
    return helpers.SqlTypeToGoSupportedTypeStr(
        col.Type, col.UdtName, col.Nullable,
    )
}

// GoTypePointer returns pointer variant for nullable fields
func GoTypePointer(col models.TableColumn) string { ... }
```

Type Mapping Examples:

PostgreSQL	Go Type	Nullable Go Type
`uuid`	`types.URID`	`*types.URID`
`integer`	`types.ID`	`*types.ID`
`text`	`string`	`*string`
`boolean`	`bool`	`*bool`
`jsonb`	`types.JSONB`	`*types.JSONB`
`timestamp`	`types.NullTime`	`*types.NullTime`
`geometry`	`types.Geometry`	`*types.Geometry`

Custom FuncMap -- GORM and Struct Tags

The `StructTags` function generates complete struct field tags:

```
func StructTagsController(col TableColumn, table Table) string
```

Output Example:

```
type User struct {
    ID        types.URID  `gorm:"primaryKey;type:uuid" json:"id"`
    Email     string      `gorm:"not null;uniqueIndex" json:"email"
                          example:"john@example.com"`
    Name      string      `gorm:"not null" json:"name"
                          example:"John Adams"`
    Status    UserStatus  `gorm:"type:user_status" json:"status"`
    CreatedAt types.NullTime `gorm:"autoCreateTime" json:"created_at"`
    UpdatedAt types.NullTime `gorm:"autoUpdateTime" json:"updated_at"`
}
```

Tags include:

- GORM directives: `primaryKey`, `not null`, `type:`, `autoCreateTime`
- JSON field names with `omitempty` for nullable fields
- Example values generated via `gofakeit` for Swagger documentation

Custom FuncMap -- Query Helpers

Functions that generate GORM query logic from schema metadata:

Function	Purpose
`PreloadFields`	Generates `db.Preload()` chains for FK relations
`GetKeyParams`	Extracts primary key parameters from Fiber context
`GetWhereClause`	Builds WHERE conditions for composite PKs
`ParsePathParams`	Generates `c.Params()` parsing for route params
`SetRequiredFields`	Lists non-nullable fields for validation
`Dict`	Creates template-local variable dictionaries

Preload Example:

```go
// For a table with FK: order.customer_id -> customers.id
// and FK: order.product_id -> products.id

func (s *OrderService) Find(id types.URID) (*Order, error) {
    var result Order
    err := s.db.
        Preload("Customer").
        Preload("Product").
        Where("id = ?", id).
        First(&result).Error
    return &result, err
```

Custom FuncMap -- Test Helpers

Functions that support test code generation:

Function	Purpose
`MockServiceMethods`	Generates mock interface implementations
`TestCreatePayload`	Builds valid JSON payloads for create tests
`TestEditPayload`	Builds valid JSON payloads for update tests
`AssertFields`	Generates field-by-field assertion statements
`RandomValue`	Produces type-appropriate random test values

These functions ensure generated tests are immediately runnable without manual fixture setup.

Template Composition

Templates use Go's `define` and `template` blocks for composition:

```
struct.go.tmpl
  |-- defines "struct_form"      (create input)
  |-- defines "struct_model"     (database model)
  |-- defines "struct_edit"      (update input)
  |-- defines "struct_filter"    (search filters)
  |-- defines "struct_identity"  (minimal reference)
  |-- defines "struct_page"      (paginated response)

controller.go.tmpl
  |-- defines "handler_create"
  |-- defines "handler_bulk_create"
  |-- defines "handler_update"
  |-- defines "handler_delete"
  |-- defines "handler_search"
  |-- defines "handler_paginate"
  |-- defines "handler_find"
  |-- includes "swagger.go.tmpl" (annotation blocks)
```

Each block is independently testable and can be overridden without affecting other parts of the template.

Template Execution Safety

Several mechanisms prevent template errors from producing invalid code:

1. **Clone Before Execute** -- Each template execution works on a clone, preventing concurrent `define` block contamination.
2. **Buffered Output** -- Templates write to `bytes.Buffer` first. Only after successful execution is the buffer flushed to disk.
3. **Post-Processing** -- Generated code passes through `goimports` to fix import ordering and remove unused imports.
4. **Type Safety in FuncMap** -- All template functions perform explicit type assertions with clear error messages on failure.
5. **Nil-Safe Access** -- Template functions handle nil pointers and empty slices gracefully, producing valid empty output rather than panicking.

Generated Code Walkthrough

Six Struct Variants Per Table

Struct Generation Example

PostgreSQL Table:

```sql
CREATE TABLE iam.users (
    id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    email       VARCHAR(255) NOT NULL UNIQUE,
    full_name   VARCHAR(100) NOT NULL,
    role_id     UUID REFERENCES iam.roles(id),
    status      iam.user_status DEFAULT 'active',
    created_at  TIMESTAMPTZ DEFAULT now(),
    updated_at  TIMESTAMPTZ DEFAULT now()
);
```

Generated Form Struct:

```go
type UserForm struct {
    Email    string      `json:"email" example:"sarah@acme.com"`
    FullName string      `json:"full_name" example:"Sarah Connor"`
    RoleID   *types.URID `json:"role_id,omitempty"`
    Status   *UserStatus `json:"status,omitempty"`
}
```

Generated Model and Edit Structs

Model (full GORM entity):

```go
type User struct {
    ID        types.URID    `gorm:"primaryKey;type:uuid" json:"id"`
    Email     string        `gorm:"not null;uniqueIndex" json:"email"`
    FullName  string        `gorm:"not null" json:"full_name"`
    RoleID    *types.URID   `gorm:"type:uuid" json:"role_id,omitempty"`
    Status    *UserStatus   `gorm:"type:user_status" json:"status,omitempty"`
    CreatedAt types.NullTime `gorm:"autoCreateTime" json:"created_at"`
    UpdatedAt types.NullTime `gorm:"autoUpdateTime" json:"updated_at"`

    // Relations (Preload)
    Role *RoleIdentity `gorm:"foreignKey:RoleID" json:"role,omitempty"`
}
```

Edit (pointer fields for partial updates):

```go
type UserEdit struct {
    Email    *string     `json:"email,omitempty"`
    FullName *string     `json:"full_name,omitempty"`
    RoleID   *types.URID `json:"role_id,omitempty"`
    Status   *UserStatus `json:"status,omitempty"`
}
```

Service Layer Generation

Each table gets an interface and an implementation:

```go
type IUserService interface {
    Create(form *UserForm) (*User, error)
    BulkCreate(forms []*UserForm) ([]*User, error)
    Update(id types.URID, edit *UserEdit) (*User, error)
    BulkUpdate(edits map[types.URID]*UserEdit) ([]*User, error)
    Delete(id types.URID) error
    Find(id types.URID) (*User, error)
}

type UserService struct {
    db *gorm.DB
}

func NewUserService(db *gorm.DB) IUserService {
    return &UserService{db: db}
}
```

The interface enables mock-based testing without database dependencies.

Service Implementation Details

Create with Outbox Pattern (when enabled):

```go
func (s *UserService) Create(form *UserForm) (*User, error) {
    record := &User{
        Email:    form.Email,
        FullName: form.FullName,
        RoleID:   form.RoleID,
        Status:   form.Status,
    }

    err := s.db.Transaction(func(tx *gorm.DB) error {
        if err := tx.Create(record).Error; err != nil {
            return err
        }
        // Outbox: persist event for async processing
        outboxEntry := OutboxEvent{
            AggregateType: "user",
            AggregateID:   record.ID.String(),
            EventType:     "user.created",
            Payload:       sonic.Marshal(record),
        }
        return tx.Create(&outboxEntry).Error
    })

    return record, err
}
```

Service: Associations and Nested Creation

When `AssociationsCreating` is enabled, the service handles parent-child relationships within a single transaction:

```go
func (s *OrderService) Create(form *OrderForm) (*Order, error) {
    return s.createWithAssociations(form)
}

func (s *OrderService) createWithAssociations(form *OrderForm) (*Order, error) {
    record := mapFormToRecord(form)

    err := s.db.Transaction(func(tx *gorm.DB) error {
        session := tx.Session(&gorm.Session{
            FullSaveAssociations: true,
        })
        return session.Create(record).Error
    })

    return record, err
}
```

Child records (e.g., `order_items`) inherit `company_id` and `created_by` from the parent context automatically via `getChildListInfos()`.

Controller Generation

Each controller wraps a service with HTTP handling:

```go
type UserController struct {
    Service IUserService
    EM      *events.EventManager  // optional
}

// @Summary Create a new user
// @Tags    IAM - Users
// @Accept  json
// @Produce json
// @Param   body body UserForm true "User data"
// @Success 201 {object} User
// @Failure 400 {object} swag.ErrorResponse
// @Router  /api/v1/iam/users [post]
func (ctrl *UserController) Create(c *fiber.Ctx) error {
    var form UserForm
    if err := c.BodyParser(&form); err != nil {
        return c.Status(400).JSON(swag.ErrorResponse{
            Message: "Invalid request body",
        })
    }

    result, err := ctrl.Service.Create(&form)
    if err != nil {
        ctrl.EM.EmitWithData(c, events.CreationError, err, result)
        return c.Status(500).JSON(swag.ErrorResponse{...})
    }

    ctrl.EM.EmitWithData(c, events.CreationSuccess, nil, result)
    return c.Status(201).JSON(result)
}
```

Controller: Pagination Endpoint

The pagination controller leverages the generic pagination engine:

```go
// @Summary  Search users with pagination
// @Tags     IAM - Users
// @Produce  json
// @Param    page     query int    false "Page number"
// @Param    size     query int    false "Page size"
// @Param    sort     query string false "Sort field"
// @Param    fields   query string false "Select fields"
// @Param    filters  query string false "Filter conditions"
// @Param    preloads query string false "Preload relations"
// @Success  200 {object} pagination.Page[User]
// @Router   /api/v1/iam/users/search [get]
func (ctrl *UserController) Paginate(c *fiber.Ctx) error {
    result, err := pagination.NewPagination[User]().
        With(ctrl.DB).
        Request(c).
        Response()

    if err != nil {
        return c.Status(500).JSON(swag.ErrorResponse{...})
    }

    return c.JSON(result)
}
```

A single generic call handles filtering, sorting, preloading, field selection, and response formatting.

Endpoint Registration

Route registration uses recursive tree grouping:

```go
func Run(app fiber.Router, db *gorm.DB,
         em *events.EventManager) {

    api := app.Group("/iam")

    // Users group
    users := api.Group("/users")
    userCtrl := &UserController{
        Service: NewUserService(db), EM: em,
    }
    users.Post("/", userCtrl.Create)
    users.Post("/bulk", userCtrl.BulkCreate)
    users.Get("/search", userCtrl.Paginate)
    users.Get("/:id", userCtrl.Find)
    users.Put("/:id", userCtrl.Update)
    users.Delete("/:id", userCtrl.Delete)

    // Nested: user_settings under /users
    settings := users.Group("/settings")
    settingsCtrl := &UserSettingController{...}
    settings.Post("/", settingsCtrl.Create)
    // ... more routes

    // SSE streams (when enabled)
    users.Get("/stream", sse.Hub.StreamResource("iam.users"))
}
```

Request Parsing

The engine parses query parameters into a structured request:

```
GET /products/search
  ?page=1
  &size=20
  &sort=price:desc,name:asc
  &fields=id,name,price,category_id
  &preloads=Category,Brand
  &filters[0][field]=price&filters[0][op]=gte&filters[0][value]=100
  &filters[1][field]=status&filters[1][op]=eq&filters[1][value]=active
  &groupBy=category_id
  &aggregations[0][function]=avg&aggregations[0][field]=price
```

Parsed into:

```go
type ParsedRequest struct {
    Page         int
    Size         int
    Sort         []SortOrder
    Fields       []string
    Preloads     []PreloadParam
    Filters      []PageFilter
    GroupBy      []string
    Aggregations []AggregationFunction
}
```

Filter System Architecture

The filter system uses the **Strategy Pattern** with three parser modes:

```
                    FilterParserComponent
                    /         |         \
        SimpleFilter    MapFilter    RecursiveFilter
        (array fmt)    (object fmt)  (nested AND/OR)
```

Simple Format:

```
filters[0][field]=price&filters[0][op]=gte&filters[0][value]=100
```

Map Format:

```
{"price": {"gte": 100}, "status": {"eq": "active"}}
```

Recursive Nested Format:

```
{
  "AND": [
    {"field": "price", "op": "gte", "value": 100},
    {"OR": [
      {"field": "status", "op": "eq", "value": "active"},
      {"field": "status", "op": "eq", "value": "pending"}
    ]}
  ]
}
```

Where Clause Builder

The `WhereClauseBuilder` maintains a registry of operator handlers.

Each operator knows how to translate a filter into a GORM `Where` clause.

Standard SQL Operators

Operator	SQL	Example
`eq`	`= ?`	`price eq 100`
`neq`	`<> ?`	`status neq deleted`
`gt` / `gte`	`> ?` / `>= ?`	`price gte 50`
`lt` / `lte`	`< ?` / `<= ?`	`price lte 500`
`in`	`IN (?)`	`status in active,pending`
`not_in`	`NOT IN (?)`	`role not_in admin,super`
`is_null`	`IS NULL`	`deleted_at is_null`
`is_not_null`	`IS NOT NULL`	`email is_not_null`
`between`	`BETWEEN ? AND ?`	`price between 10,100`
`like`	`LIKE ?`	`name like %john%`

PostgreSQL-Specific Operators

Beyond standard SQL, the engine supports PostgreSQL-native operations:

Text Search

Operator	SQL	Use Case
`ilike`	`ILIKE ?`	Case-insensitive pattern match
`fts`	`to_tsvector(?) @@ to_tsquery(?)`	Full-text search
`fts_phrase`	`to_tsvector(?) @@ phraseto_tsquery(?)`	Phrase search

Full-text search supports 16+ languages:

`english`, `turkish`, `german`, `french`, `spanish`, `arabic`, `chinese`, ...

JSONB Operators

Operator	SQL	Use Case
`jsonb_contains`	`@> ?::jsonb`	JSONB containment
`jsonb_has_key`	`? ?`	JSONB key existence
`jsonb_path`	`@? ?::jsonpath`	JSONPath query

Vector Search (pgvector)

Native support for AI/ML vector similarity search:

Operator	Distance Metric	SQL
`cosine_similarity`	Cosine	`1 - (col <=> ?)`
`l2_distance`	Euclidean	`col <-> ?`
`inner_product`	Dot Product	`col <#> ?`
`knn`	K-Nearest Neighbors	`ORDER BY col <-> ? LIMIT k`

Usage Example:

```
GET /products/search
  ?filters[0][field]=embedding
  &filters[0][op]=cosine_similarity
  &filters[0][value]=[0.1,0.2,0.3,...,0.768]
  &filters[0][threshold]=0.8
```

This enables semantic search, recommendation engines, and similarity-based retrieval directly through the pagination API.

Geospatial Search (PostGIS)

Built-in support for geographic queries:

Operator	PostGIS Function	Use Case
`st_distance`	`ST_Distance()`	Distance calculation
`st_within`	`ST_DWithin()`	Radius search
`st_intersects`	`ST_Intersects()`	Area overlap
`st_contains`	`ST_Contains()`	Containment check

Radius Search Example:

```
GET /warehouses/search
  ?filters[0][field]=location
  &filters[0][op]=st_within
  &filters[0][value]=POINT(29.0 41.0)
  &filters[0][distance]=5000
```

The `geometry_formatter.go` handles WKT (Well-Known Text) parsing for Point, LineString, Polygon, and MultiPolygon types.

Aggregation Functions

22+ aggregation functions for analytical queries:

Category	Functions
Basic	`COUNT`, `SUM`, `AVG`, `MIN`, `MAX`
Statistical	`STDDEV`, `VARIANCE`, `MEDIAN`, `MODE`
Percentile	`PERCENTILE_CONT`, `PERCENTILE_DISC`
Array	`ARRAY_AGG`, `STRING_AGG`
Boolean	`BOOL_AND`, `BOOL_OR`, `EVERY`
JSON	`JSON_AGG`, `JSONB_AGG`, `JSON_OBJECT_AGG`
Regression	`CORR`, `COVAR_POP`, `COVAR_SAMP`, `REGR_SLOPE`
Window	`ROW_NUMBER`, `RANK`, `DENSE_RANK`, `NTILE`

Example:

```
GET /orders/search
  ?groupBy=status
  &aggregations[0][function]=count&aggregations[0][field]=id
  &aggregations[1][function]=sum&aggregations[1][field]=total_amount
  &aggregations[2][function]=avg&aggregations[2][field]=total_amount
```

Preload System

Relations are preloaded with per-relation filtering and sorting:

```
GET /orders/search
  ?preloads=Customer,Items.Product
  &preloads[Customer][fields]=id,name,email
  &preloads[Items][sort]=quantity:desc
  &preloads[Items][size]=5
  &preloads[Items.Product][fields]=id,name,price
```

The engine reflects on GORM struct tags to discover associations and builds the preload chain automatically.

Schema-Based Validation prevents invalid field names:

- `strict` mode: reject unknown fields (400 error)
- `compatible` mode: silently ignore unknown fields
- `ignore` mode: pass through all fields

Response Format

```go
type Page[T any] struct {
    Items      []T   `json:"items"`
    Total      int64 `json:"total"`
    Page       int   `json:"page"`
    Size       int   `json:"size"`
    TotalPages int   `json:"total_pages"`
    HasNext    bool  `json:"has_next"`
    HasPrev    bool  `json:"has_prev"`
}
```

Collection Mode groups results by a field:

```json
{
  "collections": {
    "active": { "items": [...], "total": 42 },
    "pending": { "items": [...], "total": 7 },
    "inactive": { "items": [...], "total": 3 }
  }
}
```

Caching Layer

Built-in LRU cache with TTL for repeated queries:

```
type QueryCache interface {
    Get(key string) (interface{}, bool)
    Set(key string, value interface{}, ttl time.Duration)
    Invalidate(prefix string)
}
```

Deterministic Cache Keys are generated from the full query signature:

- Model type name
- Filter conditions (sorted)
- Sort order
- Page/size
- Selected fields
- Preload configuration

Model-Scoped Invalidation: When a record is created/updated/deleted, all cache entries for that model type are invalidated via prefix matching.

Real-Time Events and SSE

SSE Hub Implementation

```go
type Hub struct {
    subscribers map[string]map[string]chan Event
    mu          sync.RWMutex
    backplane   Backplane
    bufferSize  int
    pingInterval time.Duration
}

type Event struct {
    Topic   string
    Type    string          // "created", "updated", "deleted"
    Payload json.RawMessage
}
```

Key Characteristics:

- Thread-safe subscriber management via `sync.RWMutex`
- Non-blocking fan-out: slow consumers receive dropped events, not deadlocks
- Configurable buffer size per subscriber channel
- Automatic keep-alive pings at configurable intervals (default: 30s)
- Graceful cleanup on client disconnect

SSE Stream Handler

The SSE hub exposes a Fiber-compatible handler:

```go
func (h *Hub) StreamResource(topic string) fiber.Handler {
    return func(c *fiber.Ctx) error {
        c.Set("Content-Type", "text/event-stream")
        c.Set("Cache-Control", "no-cache")
        c.Set("Connection", "keep-alive")
        c.Set("X-Accel-Buffering", "no")

        subscriberID := uuid.New().String()
        ch := h.Subscribe(topic, subscriberID)
        defer h.Unsubscribe(topic, subscriberID)

        c.Context().SetBodyStreamWriter(func(w *bufio.Writer) {
            ticker := time.NewTicker(h.pingInterval)
            defer ticker.Stop()

            for {
                select {
                case event := <-ch:
                    fmt.Fprintf(w, "event: %s\ndata: %s\n\n",
                        event.Type, event.Payload)
                    w.Flush()
                case <-ticker.C:
                    fmt.Fprintf(w, ": ping\n\n")
                    w.Flush()
                case <-c.Context().Done():
                    return
                }
            }
        })
        return nil
    }
}
```

Backplane Interface

For multi-pod deployments, the SSE hub supports pluggable backplanes:

```
type Backplane interface {
    Publish(topic string, event Event) error
    Subscribe(topic string) (<-chan Event, error)
    Unsubscribe(topic string) error
    Close() error
}
```

Supported Implementations:

- **Redis Pub/Sub** -- Standard cross-pod broadcast
- **NATS** -- High-throughput message bus

Configuration:

```
hub := sse.NewHub(
    sse.WithBackplane(redisBackplane),
    sse.WithBufferSize(256),
    sse.WithPingInterval(15 * time.Second),
)
```

Without a backplane, events are local to the process.

Adding a backplane requires zero changes to generated code.

Generated SSE Integration

When `sseEnabled: true`, the generator produces:

1. Stream endpoint in route registration:

```
users.Get("/stream", hub.StreamResource("iam.users"))
```

2. Event emission in controllers:

```
func (ctrl *UserController) Create(c *fiber.Ctx) error {
    result, err := ctrl.Service.Create(&form)
    if err == nil {
        // Publish to SSE subscribers
        sse.Publish("iam.users", sse.Event{
            Type:    "created",
            Payload: sonic.Marshal(result),
        })
    }
    return c.Status(201).JSON(result)
}
```

Clients connect via `EventSource` and receive real-time updates for any table they subscribe to.

Event Manager System

The `EventManager` provides route-scoped lifecycle hooks for cross-cutting concerns.

Event Types (Organized in Blocks of 100)

Block	Event Types
100-199	Creation: `CreationSuccess`, `CreationError`, `CreationRequest`
200-299	Update: `UpdateSuccess`, `UpdateError`, `UpdateRequest`
300-399	Deletion: `DeletionSuccess`, `DeletionError`, `DeletionRequest`
400-499	Query: `PaginationQuerySuccess`, `SearchSuccess`, `FindSuccess`
500-599	Auth: `AuthorizationError`, `AuthenticationError`
600-699	Validation: `ValidationError`, `ParseError`

Each event carries the Fiber context, error (if any), status code, and data payload.

Event Handler Registration

Three dispatch modes for different use cases:

```go
em := events.NewEventManager()

// 1. Request interceptor (can abort the request)
em.OnRequest("/api/v1/iam/users", events.CreationRequest,
    func(e events.Event) error {
        // Validate, transform, or reject
        if !isAllowed(e.Ctx) {
            return fiber.ErrForbidden
        }
        return nil // continue processing
    },
)

// 2. Synchronous post-event handler
em.OnEvent("/api/v1/iam/users", events.CreationSuccess,
    func(e events.Event) {
        auditLog.Record(e)  // runs in request goroutine
    },
)

// 3. Asynchronous post-event handler
em.OnEventAsync("/api/v1/iam/users", events.CreationSuccess,
    func(e events.Event) {
        notificationService.Send(e)  // runs in separate goroutine
    },
)
```

Event Manager: Wildcard Handlers

Global handlers that apply to all routes:

```go
// Log every successful creation across all resources
em.OnAnyEvent(events.CreationSuccess, func(e events.Event) {
    log.Info("Resource created",
        "route", e.RoutePath,
        "status", e.StatusCode,
    )
})

// Global error tracking
em.OnAnyEvent(events.CreationError, func(e events.Event) {
    errorTracker.Capture(e.Err, map[string]string{
        "route":  e.RoutePath,
        "status": fmt.Sprint(e.StatusCode),
    })
})

// Global request audit
em.OnAnyRequest(events.CreationRequest, func(e events.Event) error {
    rateLimiter.Check(e.Ctx)
    return nil
})
```

Thread Safety: All handler registration and dispatch is protected by `sync.RWMutex` for concurrent access.

Outbox Pattern

When `outboxEnabled: true`, the generator adds transactional event persistence alongside database writes.

```
+------------------+       +------------------+
|  Service.Create  |       |   Outbox Table   |
|  (Transaction)   | ----> |   outbox_events  |
+------------------+       +------------------+
                                   |
                            Async Processor
                                   |
                            +------------------+
                            | Message Broker   |
                            | (Kafka, RabbitMQ)|
                            +------------------+
```

Guarantees:

- Business data and event are committed atomically
- No event is lost if the application crashes after commit
- Events are processed exactly-once by the outbox consumer
- Decouples synchronous API response from async processing

This eliminates the dual-write problem common in microservice architectures.

Documentation Engine

VitePress Site Generation

Each table produces a dedicated documentation page with:

1. **Schema Overview** -- Table description, row count, constraints
2. **Column Reference** -- Type mapping across SQL, Go, and TypeScript
3. **API Endpoints** -- Generated REST operations with examples
4. **Relations** -- FK references with navigation links
5. **Mermaid ER Diagram** -- Visual relationship graph

Sidebar Configuration is generated automatically:

```
[
  {
    "text": "IAM Schema",
    "items": [
      { "text": "Users", "link": "/iam/users" },
      { "text": "Roles", "link": "/iam/roles" },
      { "text": "Permissions", "link": "/iam/permissions" }
    ]
  }
]
```

Column Reference Table

For each table, the doc generator produces a comprehensive column table:

Column	SQL Type	Go Type	TS Type	Nullable	Default	Constraints
id	uuid	types.URID	string	No	gen_random_uuid()	PK
email	varchar(255)	string	string	No	-	UNIQUE
role_id	uuid	*types.URID	string?	Yes	-	FK(roles)
status	user_status	UserStatus	enum	No	'active'	ENUM
metadata	jsonb	types.JSONB	Record	Yes	'{}'	-
location	geometry	types.Geometry	GeoJSON	Yes	-	-

Three type systems (SQL, Go, TypeScript) are presented side-by-side, eliminating the need for manual cross-referencing.

API Documentation Tabs

Each table page includes tabbed API documentation:

Tab: Search

```
GET /api/v1/{schema}/{table}/search
  ?page=1&size=20&sort=created_at:desc
  &filters[0][field]=status&filters[0][op]=eq&filters[0][value]=active

Response: Page[Model]
```

Tab: Create

```
POST /api/v1/{schema}/{table}
Content-Type: application/json

Body: FormStruct (required fields highlighted)
Response: 201 Model
```

Tab: Find / Update / Delete

```
GET    /api/v1/{schema}/{table}/:id  -> Model
PUT    /api/v1/{schema}/{table}/:id  -> Model (EditStruct body)
DELETE /api/v1/{schema}/{table}/:id  -> 204 No Content
```

TypeScript Documentation

The doc generator produces frontend-ready TypeScript artifacts:

Interfaces:

```
interface User {
  id: string;
  email: string;
  full_name: string;
  role_id?: string;
  status?: UserStatus;
  created_at: string;
  updated_at: string;
}

type UserForm = Omit<User, 'id' | 'created_at' | 'updated_at'>;
type UserEdit = Partial<UserForm>;
```

React Query Hooks:

```
const useUsers = (params?: PaginationParams) =>
  useQuery(['users', params], () => fetchUsers(params));

const useCreateUser = () =>
  useMutation((form: UserForm) => createUser(form));
```

Zod Validation Schemas:

```
const UserFormSchema = z.object({
  email: z.string().email(),
```

MCP Knowledge Graph Export

Model Context Protocol export for AI agent consumption:

Three JSONL Files

graph_nodes.jsonl -- Every entity in the schema:

```
{"id": "iam.users", "type": "table", "schema": "iam",
 "name": "users", "columns": 7, "pk": "id",
 "description": "User accounts and authentication"}
```

graph_edges.jsonl -- Relationships between entities:

```
{"source": "iam.users", "target": "iam.roles",
 "type": "belongs_to", "via": "role_id",
 "cardinality": "many_to_one"}
```

doc_chunks.jsonl -- Semantic text chunks for RAG:

```
{"content": "The users table stores authentication...",
 "metadata": {"schema": "iam", "table": "users",
              "section": "overview"}}
```

manifest.json -- Index with node/edge/chunk counts.

Training Data Generation

JSONL dataset for LLM fine-tuning with system/user/assistant triplets:

```
{
  "messages": [
    {"role": "system", "content": "You are an API documentation expert..."},
    {"role": "user", "content": "What columns does the users table have?"},
    {"role": "assistant", "content": "The users table has 7 columns: id (UUID PK), email (VARCHAR UNIQUE), full_name (VARCHAR), role_id (UUID FK->roles), status (ENUM: active|inactive|suspended), created_at (TIMESTAMPTZ), updated_at (TIMESTAMPTZ)."}
  ]
}
```

Generated Question Categories

Category	Example Questions
Schema QA	"What tables are in the IAM schema?"
Column Info	"What type is the users.status column?"
Endpoint Usage	"How do I create a new user?"
Filter Examples	"How do I search for active users?"
React Query	"How do I fetch paginated products?"
Zod Schemas	"What validation does UserForm need?"
RPC Usage	"How do I call the calculate_totals function?"

Mermaid ER Diagrams

Automatically generated entity-relationship diagrams:

```
erDiagram
    USERS {
        uuid id PK
        varchar email UK
        varchar full_name
        uuid role_id FK
        user_status status
        timestamptz created_at
        timestamptz updated_at
    }
    ROLES {
        uuid id PK
        varchar name UK
        varchar slug UK
    }
    PERMISSIONS {
        uuid id PK
        varchar name UK
        varchar resource
        varchar action
    }
    ROLE_PERMISSIONS {
        uuid role_id FK
        uuid permission_id FK
    }

    USERS ||--o{ ROLES : "role_id"
    ROLE_PERMISSIONS }o--|| ROLES : "role_id"
    ROLE_PERMISSIONS }o--|| PERMISSIONS : "permission_id"
```

Source Linker

The `SourceLinker` generates GitHub permalink badges for each generated file, connecting documentation to source code:

```go
type SourceLinker struct {
    GitHubRepo string  // "github.com/org/repo"
    Branch     string  // "main"
    BasePath   string  // "api_v2"
}

func (sl *SourceLinker) StructLink(schema, table string) string {
    return fmt.Sprintf(
        "[![Source](https://img.shields.io/badge/source-GitHub-blue)]"+
        "(https://%s/blob/%s/%s/%s/structures/%s.go)",
        sl.GitHubRepo, sl.Branch, sl.BasePath, schema, table,
    )
}
```

Every documentation page includes direct links to:

- Struct definition file
- Service implementation file
- Controller handler file
- Endpoint registration file

Testing and Quality Assurance

Generated Unit Tests

For each table controller, the generator produces unit tests with mock service implementations:

```go
type MockUserService struct {
    mock.Mock
}

func (m *MockUserService) Create(form *UserForm) (*User, error) {
    args := m.Called(form)
    return args.Get(0).(*User), args.Error(1)
}

func TestUserController_Create(t *testing.T) {
    mockService := new(MockUserService)
    ctrl := &UserController{Service: mockService}

    expectedUser := &User{ID: types.NewURID(), Email: "test@example.com"}
    mockService.On("Create", mock.AnythingOfType("*UserForm")).
        Return(expectedUser, nil)

    app := fiber.New()
    app.Post("/users", ctrl.Create)

    req := httptest.NewRequest("POST", "/users",
        strings.NewReader(`{"email":"test@example.com"}`))
    req.Header.Set("Content-Type", "application/json")

    resp, _ := app.Test(req)
    assert.Equal(t, 201, resp.StatusCode)
}
```

Generated Integration Tests

Full HTTP round-trip tests against a real database:

```go
func TestUserAPI_Integration(t *testing.T) {
    // Setup: real DB + real Fiber app
    db := setupTestDB(t)
    app := fiber.New()
    service := NewUserService(db)
    ctrl := &UserController{Service: service}
    app.Post("/users", ctrl.Create)
    app.Get("/users/:id", ctrl.Find)

    // Test Create
    createResp, _ := app.Test(httptest.NewRequest("POST", "/users",
        strings.NewReader(`{
            "email": "integration@test.com",
            "full_name": "Integration User"
        }`)))
    assert.Equal(t, 201, createResp.StatusCode)

    var created User
    json.NewDecoder(createResp.Body).Decode(&created)
    assert.NotEmpty(t, created.ID)

    // Test Find
    findResp, _ := app.Test(httptest.NewRequest("GET",
        "/users/"+created.ID.String(), nil))
    assert.Equal(t, 200, findResp.StatusCode)
}
```

Generated Test Helpers

Shared infrastructure for all test files within a schema:

```go
// test_helpers.go -- generated per schema

func setupTestDB(t *testing.T) *gorm.DB {
    dsn := os.Getenv("TEST_DATABASE_URL")
    db, err := gorm.Open(postgres.Open(dsn), &gorm.Config{})
    require.NoError(t, err)

    // Auto-migrate all schema tables
    db.AutoMigrate(&User{}, &Role{}, &Permission{}, ...)

    t.Cleanup(func() {
        sqlDB, _ := db.DB()
        sqlDB.Close()
    })
    return db
}

func createTestUser(t *testing.T, db *gorm.DB) *User {
    user := &User{
        Email:    gofakeit.Email(),
        FullName: gofakeit.Name(),
    }
    require.NoError(t, db.Create(user).Error)
    return user
}
```

E2E Test Pipeline

The `app/e2e_test.go` validates the entire generation pipeline:

```
1. Spin up PostgreSQL      Docker container with test schemas
       |
2. Write test config       Temporary config.yaml with all features
       |
3. Run generator           app.Run() with test config
       |
4. Assert file existence   Check all expected output files
       |
5. Initialize Go module    go mod init + go mod tidy
       |
6. Compile generated code  go build ./...
       |
7. Run generated tests     go test ./... on generated test suite
       |
8. Cleanup                 Remove temp files, stop Docker
```

This ensures that template changes never produce code that fails to compile or whose generated tests fail.

Generator Unit Tests

The generator itself has comprehensive tests for each component:

Template Helper Tests

```go
func TestToCamel(t *testing.T) {
    tests := []struct{ input, expected string }{
        {"user_settings", "UserSettings"},
        {"api_v2_config", "ApiV2Config"},
        {"id", "ID"},
        {"url", "URL"},
    }
    for _, tt := range tests {
        assert.Equal(t, tt.expected, ToCamel(tt.input))
    }
}
```

Creator Tests

```go
func TestControllerCreator_Create(t *testing.T) {
    table := models.Table{Name: "users", Columns: testColumns}
    creator := &ControllerCreator{Context: testContext}

    output, err := creator.CreateTableController(table, "iam")
    require.NoError(t, err)

    assert.Contains(t, output, "func (ctrl *UserController) Create")
    assert.Contains(t, output, "@Router /api/v1/iam/users [post]")
    assert.Contains(t, output, "c.BodyParser")
}
```

Pagination Engine Tests

The pagination package has the most extensive test suite: 20+ test files covering every operator, aggregation, and edge case.

Test File	Coverage
`core_test.go`	Basic pagination flow
`executor_test.go`	Query execution paths
`filter_test.go`	All filter operators
`recursive_filter_test.go`	Nested AND/OR logic
`aggregation_test.go`	All 22+ aggregation functions
`vector_search_test.go`	pgvector operators
`geo_search_test.go`	PostGIS operators
`search_language_test.go`	Full-text search languages
`lru_cache_test.go`	Cache hit/miss/eviction/TTL
`relations_integration_test.go`	Preload with nested relations
`query_builder_test.go`	GORM reflection and schema mapping
`benchmark_test.go`	Performance benchmarks

Quality Gates

```
Developer Workflow:
                                                Merge
  Code Change --> Unit Tests --> E2E --> Compile --> Deploy
       |               |          |          |
       |          Template    Full        go build
       |          helpers    pipeline     ./...
       |          Creators   Docker PG
       |          Pagination
       |
       v
  [Generator Change]
       |
       v
  Re-generate all projects
       |
       v
  Compile + test generated output
```

No template change ships without:

1. Generator unit tests passing
2. E2E pipeline completing (Docker PostgreSQL)
3. Generated code compiling cleanly
4. Generated tests passing

This feedback loop ensures that the generator and its output remain in sync at all times.

Worker Pool Architecture

The `WorkerPool` provides bounded concurrency with panic recovery:

```go
type WorkerPool struct {
    workers    int
    jobQueue   chan Job
    results    chan Result
    wg         sync.WaitGroup
    ctx        context.Context
    cancel     context.CancelFunc
}

type Job struct {
    ID       int
    Execute  func() (interface{}, error)
}

type Result struct {
    JobID    int
    Output   interface{}
    Error    error
    Duration time.Duration
}
```

Default workers: `runtime.NumCPU()`

Each worker runs in its own goroutine with `defer recover()` to prevent a single table failure from crashing the entire run.

Parallel Table Generation

The `ParallelCreator` distributes table processing across workers:

```
                Schema: "iam" (11 tables)
                          |
                ParallelCreator.Generate()
                          |
           +-------+-------+-------+-------+
           |       |       |       |       |
        Worker1  Worker2  Worker3  Worker4  ...
        users    roles    perms   sessions
           |       |       |       |
         struct  struct  struct  struct
         service service service service
         ctrl    ctrl    ctrl    ctrl
         test    test    test    test
           |       |       |       |
           +-------+-------+-------+
                          |
                All files written
                          |
                EndPointCreator (sequential)
```

Struct, service, controller, and test files for different tables are independent -- they can safely run in parallel.

Route registration runs sequentially after all tables complete because it aggregates all table metadata into a single file.

Object Pooling

Three `sync.Pool` implementations reduce GC pressure:

FileWriterPool

```go
var writerPool = sync.Pool{
    New: func() interface{} {
        return bufio.NewWriterSize(nil, 64*1024) // 64KB buffer
    },
}
```

Reuses buffered writers across file generation calls.

Directory existence is cached to avoid redundant `os.MkdirAll` calls.

StringBuilderPool

```go
var builderPool = sync.Pool{
    New: func() interface{} {
        return &strings.Builder{}
    },
}

func FastStringConcat(parts ...string) string {
    b := builderPool.Get().(*strings.Builder)
    defer builderPool.Put(b)
    b.Reset()
    // Pre-calculate total length
    totalLen := 0
    for _, p := range parts {
        totalLen += len(p)
```

Template Cache Performance

Templates are parsed once and cloned for each execution:

```
Startup:
  Parse 13 templates -> Cache (sync.Once)

Per Table Execution:
  1. template.Clone()        ~50ns (vs ~5ms parse)
  2. Execute(clone, data)    ~1ms
  3. Write to buffered I/O   ~0.5ms

Total per table: ~1.5ms (vs ~6.5ms without cache)
Speedup: 4.3x per table operation
```

For a schema with 50 tables generating 4 files each,

the cache saves approximately **1 second** of template parsing time.

Performance Monitor

The `PerformanceMonitor` tracks real-time metrics during generation:

```
type PerformanceMonitor struct {
    operations map[string]*OperationMetrics
    mu         sync.RWMutex
    startTime  time.Time
}

type OperationMetrics struct {
    Count       int64
    TotalTime   time.Duration
    MinTime     time.Duration
    MaxTime     time.Duration
    MemoryDelta int64
}
```

Tracked Operations:

- Schema introspection time
- Per-table struct/service/controller generation
- Template execution duration
- File I/O duration
- Total memory allocation

Benchmark Results

Measured on a schema with 50+ tables across 5 schemas:

Sequential vs Parallel Generation

Mode	Tables	Duration	Files	Speed
Sequential	54	~8.2s	312	38 files/s
Parallel (8 workers)	54	~2.4s	312	130 files/s

Speedup: 3.4x with NumCPU workers.

Per-Operation Breakdown

Operation	Avg Duration	Percentage
Schema Introspection	120ms	5%
Template Execution	0.8ms/file	35%
File I/O	0.4ms/file	15%
goimports Post-Processing	1.1s	45%

The bottleneck is `goimports` post-processing, which runs sequentially across all generated files.

Memory Efficiency

Allocation Strategy

Technique	Savings
`sync.Pool` for writers	~60% fewer allocations per file
`sync.Pool` for builders	~40% fewer string allocations
Template clone vs re-parse	~90% fewer template allocations
Pre-sized slice/map init	~30% fewer slice grows
`sonic.Marshal` vs `json`	~50% faster JSON encoding

Memory Profile (54 tables, 312 files)

```
Before optimization:
  Total Alloc: 285 MB
  Num GC:      142

After optimization:
  Total Alloc: 127 MB   (-55%)
  Num GC:       61       (-57%)
```

Concurrency Safety

All concurrent access points are protected:

Resource	Protection
Template Cache	`sync.RWMutex` + `sync.Once`
File Writer Pool	`sync.Pool` (inherently safe)
String Builder Pool	`sync.Pool` (inherently safe)
SSE Subscribers	`sync.RWMutex`
Event Handlers	`sync.RWMutex`
Performance Monitor	`sync.RWMutex`
Worker Pool Jobs	Buffered channels
Project Semaphore	Buffered channel (cap 2)

No global mutable state.

All state is either immutable (`GeneratorContext`), pool-managed, or mutex-protected. This enables safe concurrent project generation without race conditions.

Generated Output

By the Numbers

Artifact	Count
Go source files	312
Documentation pages	121
OpenAPI paths	214
OpenAPI schemas	359
MCP graph nodes	628
MCP graph edges	723
MCP doc chunks	235
Training data entries	~500

Generated Directory Structure

```
api_v2/
  api.go              Main aggregation (wires all schemas)
  shared_types.go     Cross-schema FK identity structs
  iam/                IAM schema package
  catalog/            Catalog schema package
  orders/             Orders schema package
  logistics/          Logistics schema package
  analytics/          Analytics schema package
```

Per-Schema Package Structure

Each schema follows an identical package layout:

```
iam/
  iam.go                      Route registration (Run function)
  structures/
    iam_enums.go              Typed enum constants
    users.go                  User structs (Form, Model, Edit, ...)
    roles.go                  Role structs
    ... (11 files)
  services/
    users_service.go          CRUD + bulk operations
    roles_service.go
    ... (11 files)
  controllers/
    users_controller.go       Fiber handlers + Swagger
    roles_controller.go
    ... (11 files)
  tests/
    users_test.go             Unit tests with mocks
    ...
  testhelper/
    test_helpers.go           Shared test infrastructure
  pkg/iam_events/
    event_handlers.go         Event handler stubs
  outbox/
    users_outbox.go           Outbox event definitions
```

IAM Schema Highlights

Core Tables:

Table	Features Demonstrated
users	UUID PK, ENUM status, FK to roles, unique email
roles	Hierarchical (parent_id self-ref), slug unique
permissions	Composite unique (resource + action)
role_permissions	Junction table, composite FK
user_sessions	TTL-based, JSON metadata
audit_logs	Append-only, JSONB payload
api_keys	Hashed secret, expiration
user_preferences	Key-value JSONB store
teams	Hierarchical with avatar URL
team_members	Junction with role ENUM
invitations	Expiration + ENUM status

Catalog Schema Highlights

Advanced Data Types:

Table	Notable Features
categories	Self-referential hierarchy (parent_id)
products	JSONB metadata, text[] tags, vector embedding
product_variants	Multi-attribute variants (size, color, material)
product_images	S3 URL references, sort ordering
price_lists	Multi-currency, date-range validity
inventory	Warehouse-product junction, reorder tracking
suppliers	JSONB contact info, rating system
brands	Slug-based routing, logo URL
reviews	Star rating, verified purchase flag
wishlists	User-product junction, privacy setting
collections	Curated product groupings

Demonstrates: JSONB queries, array fields, self-referential FKs, multi-table joins, and soft-delete patterns

Orders Schema Highlights

Transactional Patterns:

Table	Notable Features
orders	Status state machine, total calculation
order_items	Composite FK (order + variant), quantity/price
payments	Gateway integration, idempotency key
refunds	Parent payment reference, partial amounts
coupons	Usage limits, date validity, discount types
order_coupons	Junction with discount tracking
order_status_history	Append-only state transitions
shipping_addresses	Structured address with coordinates
order_notes	Internal/external visibility flag

Demonstrates: Transactional outbox, nested creation (order + items), state machine tracking, and monetary precision handling.

Logistics and Analytics Highlights

Logistics Schema

- **Warehouses**: PostGIS geometry for location
- **Shipments**: Multi-carrier tracking, ETA calculation
- **Routes**: Optimized delivery path with waypoints
- **Vehicles**: Fleet management with capacity constraints

Demonstrates: Geospatial queries (ST_Distance, ST_Within), `relaxGrouping` for flat route tree, real-time SSE tracking.

Analytics Schema

- **Dashboards**: Configurable layout with widget grid
- **Reports**: Scheduled generation with template system
- **KPI Definitions**: Formula-based metrics
- **Data Sources**: Multi-database connector config
- **SQL Views**: `daily_revenue_summary`, `product_performance`
- **Materialized Views**: Pre-aggregated analytics data

Demonstrates: Views and materialized views handling, aggregation queries, JSONB configuration storage.

RPC Function Examples

Stored procedures generated as typed API endpoints:

PostgreSQL:

```
CREATE OR REPLACE FUNCTION orders.rpc__calculate_order_total(
    p_order_id UUID
) RETURNS NUMERIC AS $$
    SELECT COALESCE(SUM(quantity * unit_price), 0)
    FROM orders.order_items
    WHERE order_id = p_order_id;
$$ LANGUAGE sql STABLE;
```

Generated Go Service:

```
func (s *RPCService) CalculateOrderTotal(orderID types.URID) (float64, error) {
    var result float64
    err := s.db.Raw("SELECT orders.rpc__calculate_order_total(?)",
        orderID).Scan(&result).Error
    return result, err
}
```

Generated Endpoint:

```
POST /api/v1/orders/rpc/calculate-order-total
Body: {"order_id": "uuid-here"}
Response: {"result": 1299.99}
```

Documentation Site

The generated VitePress documentation site includes:

1. **Landing Page** -- Project overview with schema feature cards
2. **Per-Table Pages** -- Column reference, API docs, ER diagrams
3. **Swagger UI** -- Interactive OpenAPI 3.0 explorer (vitepress-openapi)
4. **Schema READMEs** -- Architecture overview per schema
5. **API Reference** -- Aggregated endpoint listing
6. **Missing Comments Report** -- Columns lacking documentation

Deployment

```yaml
# GitHub Actions workflow included
name: Deploy Documentation
on:
  push:
    branches: [main]

# Builds VitePress + deploys to GitHub Pages
# URL: https://{org}.github.io/{repo}/
```

Main Application Entry Point

```go
func main() {
    // Database connection
    db, err := gorm.Open(postgres.Open(dsn), &gorm.Config{})

    // Event Manager
    em := events.NewEventManager()

    // Fiber HTTP server
    app := fiber.New(fiber.Config{
        JSONEncoder: sonic.Marshal,
        JSONDecoder: sonic.Unmarshal,
    })

    // Middleware
    app.Use(cors.New())
    app.Use(logger.New())
    app.Use(recover.New())

    // Health check
    app.Get("/health", func(c *fiber.Ctx) error {
        return c.JSON(fiber.Map{"status": "ok"})
    })

    // Mount all generated APIs
    apiGroup := app.Group("/api/v1")
    api.Setup(apiGroup, db, em)

    // Start server
    app.Listen(":3000")
}
```


Summary

Design Pattern Summary

Pattern	Where Applied
Singleton	Template cache, file writer pool, string builder pool, SSE hub
Strategy	Filter parsers (simple/map/nested), where clause operators
Worker Pool	Parallel table generation with bounded goroutines
Builder / Fluent API	Pagination: `NewPagination().With(db).Request(c).Response()`
Template Method	All creators: prepare data, execute template, write file
Fan-Out	SSE broadcast, EventManager dispatch
Object Pool	`sync.Pool` for writers and string builders
Functional Options	SSE hub config, EventManager config, pagination config
Recursive Tree	Route hierarchy from table naming conventions
Transactional Outbox	Atomic data + event persistence

Component Metrics

Component	Files	Lines of Code	Key Capability
Parser Engine	15	~4,000	Schema introspection + code gen
Pagination	30+	~3,000	Generic query engine
Templates	13	~2,500	Code shape definitions
Doc Generator	12	~2,000	Multi-format documentation
SSE Package	3	~500	Real-time streaming
Events Package	3	~400	Lifecycle event hooks
Helpers	5	~600	Type mapping + I/O
Models	2	~300	Data structures
App / Config	5	~800	Orchestration
Total	**88+**	**~14,100**	

Differentiators

What Sets DataBridge V2 Apart

1. Depth of Generation

Not just CRUD scaffolding. Full service layer with bulk operations, transaction management, outbox pattern, and association handling.

2. Advanced Query Engine

22+ aggregation functions, vector search (pgvector), geospatial queries (PostGIS), recursive filters, multi-language full-text search.

3. Real-Time Built In

SSE streaming and event hooks are first-class features, not afterthoughts. Backplane support for multi-pod production deployments.

4. Documentation as Code

Six documentation formats generated from the same schema. MCP knowledge graphs enable AI agent consumption.

5. Quality Assurance Pipeline

Generated tests, E2E validation, performance benchmarks.

Extensibility

Adding a New Output Type

To add a new generated file type (e.g., GraphQL resolvers):

1. Create a new `.go.tmpl` template in `templates/gocode/`
2. Create a new `*_creator.go` in `parser/`
3. Add the call in `parser/creator.go` Parse() pipeline
4. Add configuration flag in `app/config.go` if optional

No changes needed to:

- Schema introspection
- Type mapping
- File writing infrastructure
- Template cache
- Parallel execution

The architecture is designed for additive extension without modifying existing components.

Supported PostgreSQL Features

Feature	Support Level
Tables (BASE TABLE)	Full
Views	Full
Materialized Views	Full
ENUMs	Full (typed Go constants)
Stored Procedures (rpc__)	Full (service + controller)
Composite Primary Keys	Full
Foreign Keys	Full (preload relations)
Unique Constraints	Full (GORM uniqueIndex)
Self-Referential FKs	Full (hierarchical trees)
JSONB Columns	Full (typed + query operators)
Array Columns	Full (text[], int[])
Vector Columns (pgvector)	Full (similarity search)
Geometry Columns (PostGIS)	Full (spatial queries)
Timestamps with Auto-Set	Full (autoCreateTime/autoUpdateTime)

Future Roadmap

Planned Enhancements

Short Term

- GraphQL schema generation alongside REST
- gRPC protobuf generation for internal services
- Custom middleware injection points in generated controllers
- Configurable response envelope format

Medium Term

- Multi-database support (MySQL, SQLite introspection)
- Real-time schema diffing and incremental regeneration
- Generated OpenTelemetry instrumentation
- WebSocket support alongside SSE

Long Term

- Visual schema designer with live preview
- AI-assisted schema optimization recommendations
- Automated migration generation from schema changes
- Multi-language output (TypeScript/Python backends)

Thank You

DataBridge V2

From PostgreSQL Schema to Production-Ready API

In a Single Command

Maple Technologies -- Engineering Division

For questions, documentation, and source code:

github.com/maple-tech/backend-generator/apiv2